

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1412

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO2: *Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 25

**Duration :60 Mins
On Class
Total Marks:50**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Bhavya S(192524376) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Bhavya S

NAME : S.BHAVYA

COURSE CODE : CSA1412

REG NO : 192524376

COURSE NAME : COMPILER DESIGN

C02-AT2

1. Parser Validation of $id + id * id$

→ The parser checks whether the token sequence follows the grammar rules.

→ It, ^{1st} recognizes id , then $*$ has higher precedence than $+$, so it parses $id * id$ before combining it with the ^{1st} id using $+$.

→ If the entire input matches the grammar & no tokens remain, the expression is syntactically valid.

2. Derivation for $aaabbb$

Grammar:

$$S \rightarrow aSb \mid \epsilon$$

Left Most Derivation:

$$\begin{aligned} S & \\ \Rightarrow aSb & \\ \Rightarrow aaSbb & \\ \Rightarrow aaasbbb & \\ \Rightarrow aaa\epsilon bbb & \\ \Rightarrow aaa.bbb & \end{aligned}$$

Hence, $aaabbb$ belongs to the language.

3. Remove left recursion:

Given:

$$E \rightarrow E + T \mid T$$

After removing left recursion:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

4. $\text{First}(A)$:

Grammar:

$$A \rightarrow aB \mid \epsilon$$

$$B \rightarrow b$$

$$\text{First}(A) = \{a, \epsilon\}$$

Since one production starts with a & the other is ϵ .

5. Grammar for top-down parsing?

Grammar:

$$A \rightarrow Aa \mid b$$

No, the grammar contains left recursion, which causes infinite recursion in recursive descent parsing. Therefore, it is not suitable for top-down ($LL(1)$) parsing.

6. Apply Left factoring:

Given:

$$S \rightarrow \text{while } E \text{ do } S \mid \text{while } E \text{ do } S \text{ else } S$$

After left factoring:

$$S \rightarrow \text{while } E \text{ do } S S'$$

$$S' \rightarrow \text{else } S \mid \epsilon$$

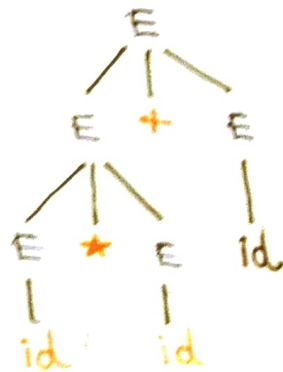
This removes the common prefix $\text{while } E \text{ do } S$, making the grammar suitable for predictive parsing.

7. Parse tree for $id * id + id$

Grammar:

$E \rightarrow E + E \mid E * E \mid id$

Parse tree:



This represents $(id * id) + id$.

8. Recursive descent procedures:

Grammar:

$S \rightarrow aA$

$A \rightarrow bA \mid c$

Procedure S()

```
{  
  match('a');  
  A();  
}
```

Procedure A()

```
{  
  if (next == 'b')  
  {  
    match('b');  
    A();  
  }  
  else  
    match('c');  
}
```

9. Predictive Parsing Table:

Grammar:

$$S \rightarrow AB$$

$$A \rightarrow a | \epsilon$$

$$B \rightarrow b$$

$$\text{First}(A) = \{a, \epsilon\}$$

$$\text{Follow}(A) = \{b\}$$

Non-Terminal	a	b	\$
S	$S \rightarrow AB$	$S \rightarrow AB$	-
A	$A \rightarrow a$	$A \rightarrow \epsilon$	-
B	-	$B \rightarrow b$	-

10. Shift-Reduce Parser for $\text{id} + \text{id}$:

Step	Stack	Input	Action
1	id	$+ \text{id} \$$	Shift id
2	E	$+ \text{id} \$$	Reduce $\text{id} \rightarrow E$
3	$E +$	$\text{id} \$$	Shift +

11. Handle Expression $(\text{id} * \text{id} + \text{id})$:

Grammar:

$$E \rightarrow E + E | E * E | \text{id}$$

The handle is a substring that is reduced during bottom-up parsing.

→ First handle: id (rightmost id)

→ Then $E * E$

→ Finally $E + E$

∴ The first handle is id .

12. Operator Precedence Relations (+, *):

Since * has higher precedence than +:

Operator Pair	Relation
$+, +$	$=$
$*, *$	$=$
$+, *$	$<$
$*, +$	$>$

∴ Multiplication is performed before addition

13. Operator Precedence parsing handles $id + id * id$:

The parser uses precedence rules to determine the order of operations:

Read the expression $id + id * id$.

Since * has higher precedence than +, it 1st reduces $id * id$.

Then it performs the addition with the remaining id .

Expression is parsed as:

$id + (id * id)$

14. LR Parser:

→ LR Parser are bottom-up parsers that read the input from left to right & produce the Rightmost derivation in reverse.

→ They use shift & reduce operations to parse the input efficiently.

Eg:

Input: $id + id :$

1. Shift id
2. Reduce $id \rightarrow E$
3. Shift $+$
4. Shift id
5. Reduce $id \rightarrow E$
6. Reduce $E + E \rightarrow E$

The input is accepted.

15. SLR Parsing Table:

Grammar:

$S \rightarrow AA$

$A \rightarrow aA \mid b$

Steps:

1. Construct the augmented grammar.
2. Find the LR(0) item sets using closure & goto.
3. Draw the LR(0) automation.
4. Compute FIRST & FOLLOW sets.
5. Fill the ACTION table using shift & reduce actions.
6. Fill the GOTO table for non-terminals.
7. Check for conflicts; if none exist, the SLR parsing table is complete.

16. Follow(A):

Grammar:

$S \rightarrow AB$

$A \rightarrow a \mid \epsilon$

$B \rightarrow b$

Since A is followed by B in $S \rightarrow AB$.

$\text{First}(B) = \{b\} \therefore \text{Follow}(A) = \{b\}$

17. Augmented Grammar:

Grammar:

$$S \rightarrow aA$$

$$A \rightarrow b$$

Augmented Grammar:

$$S' \rightarrow S$$

$$S \rightarrow aA$$

$$A \rightarrow b$$

18. Closure of the LR(0) item:

Given Item:

$$S' \rightarrow \cdot S$$

Grammar:

$$S \rightarrow aA$$

$$A \rightarrow b$$

Closure:

$$S' \rightarrow \cdot S$$

$$S \rightarrow \cdot aA$$

19. To Check whether the Grammar is Ambiguous:

Grammar:

$$E \rightarrow E + E \mid id$$

String:

$$id + id + id$$

This grammar is ambiguous bc the string has 2 different parse trees:

1. $(id + id) + id$

2. $id + (id + id)$

∴ The grammar is ambiguous.

20. Left - Most Derivation for $id + id * id$

Grammar:

$$E \rightarrow E + E \mid E * E \mid id$$

Left - Most Derivation:

E

$$\Rightarrow E + E$$

$$\Rightarrow id + E$$

$$\Rightarrow id + E * E$$

$$\Rightarrow id + id * E$$

$$\Rightarrow id + id * id$$

21. Apply Left Factoring:

Given:

$$S \rightarrow aBc \mid aBd \mid b$$

After Left Factoring:

$$S \rightarrow aBs' \mid b$$

$$s' \rightarrow c \mid d$$

22. First (F):

Grammar:

$$F \rightarrow (E) \mid id$$

Since 1 production begins with (& the other with id:

$$\text{First}(F) = \{ (, id \}$$

23. Predictive Parsing:

Grammar:

$$S \rightarrow aA \mid aB$$

$$A \rightarrow b$$

$$B \rightarrow c$$

Answer: No

Justification:

→ Both productions of S begin with the same terminal a .

→ Therefore,

$$\text{First}(aA) = \{a\}$$

$$\text{First}(aB) = \{a\}$$

→ Since the FIRST set overlap, the parser cannot choose the correct production using one lookahead symbol.

→ Left factoring is required, so the grammar is not suitable for predictive ($LL(1)$) parsing.

24. Recursive Descent Parser. Pseudocode:

Grammar:

$$S \rightarrow abS \mid \epsilon$$

Procedure $S()$

```
{
  if (next == 'a')
  {
    match('a');
    match('b');
    S();
  }
  else
    return;
}
```

25. Predictive Parsing table:

Grammar:

$$S \rightarrow aS \mid b$$

$$\text{First}(S) = \{a, b\}$$

Non-terminal	a	b	\$
S	$S \rightarrow aS$	$S \rightarrow b$	-

Since, there is no ϵ -production, the \$ entry remains empty (error).